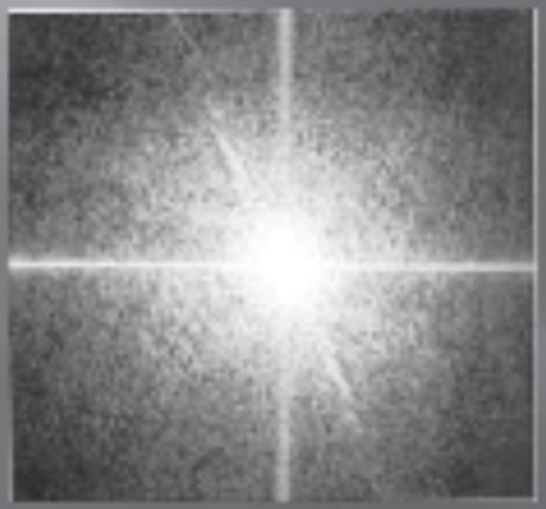
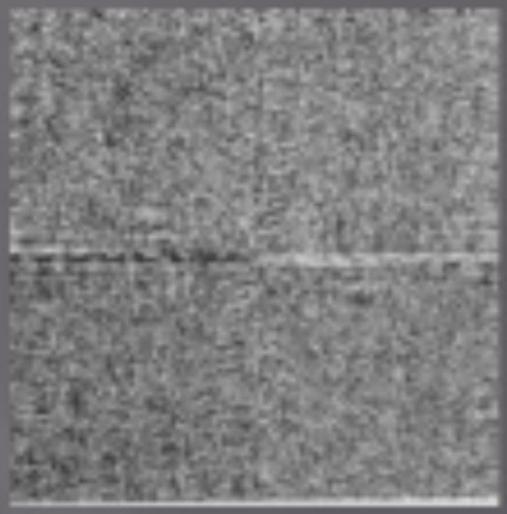




magnitude



phase

Digital Image Processing
Lecture 6: Filtering in Frequency Domain

Dr.Wafaa Shalash

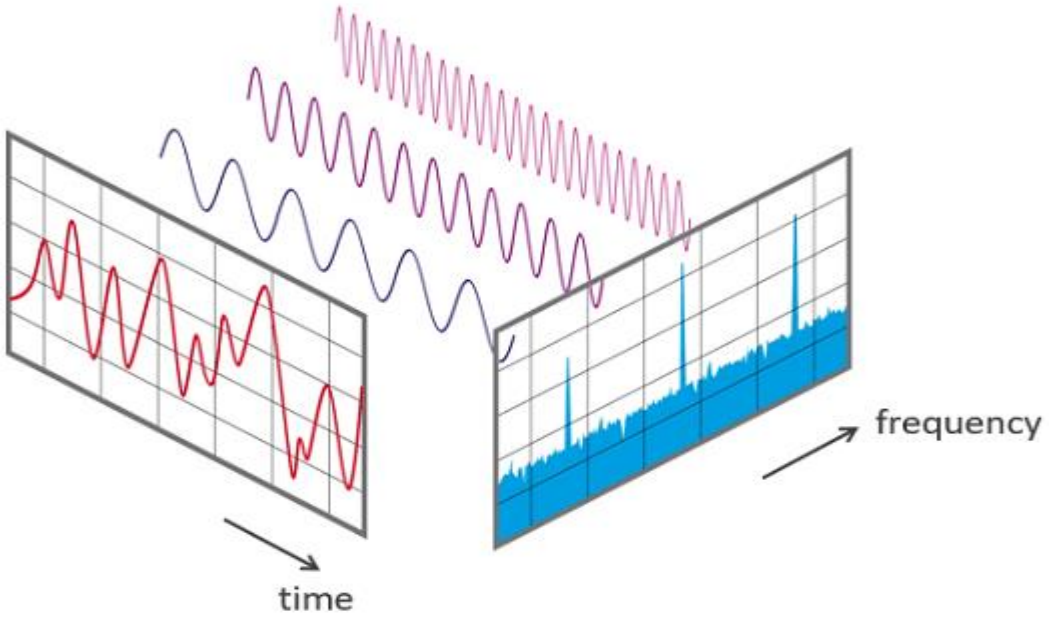

REMEMBER

Introduction

Jean Baptiste Joseph Fourier

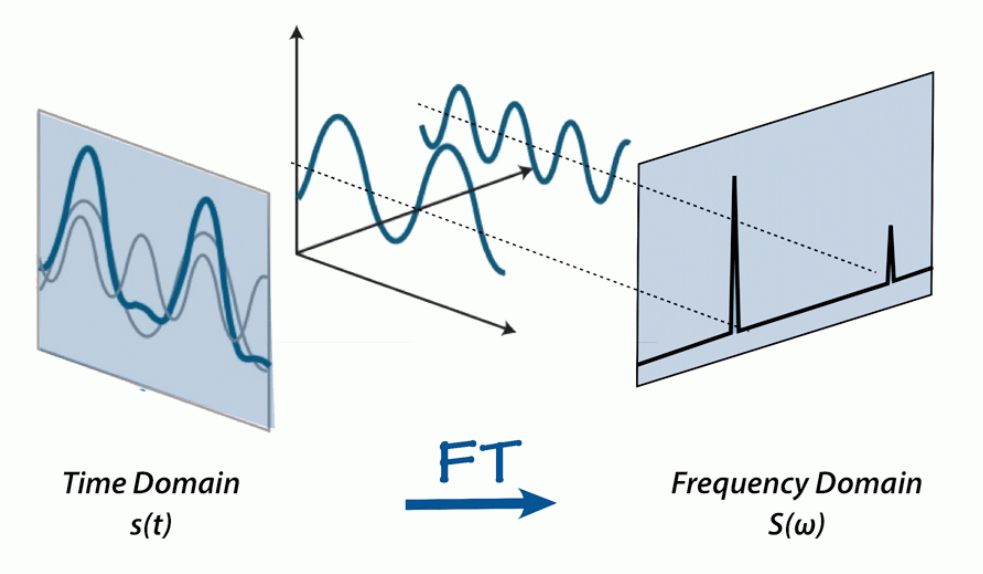


- The Fourier Transform is generally used to decompose a signal into various sinusoidal components.
- For an image, the output of the transformation is the representation of the image in frequency space, while the input image is the real space equivalent.
- In the Fourier space image, each point represents a particular frequency contained in the real domain image.



Significance

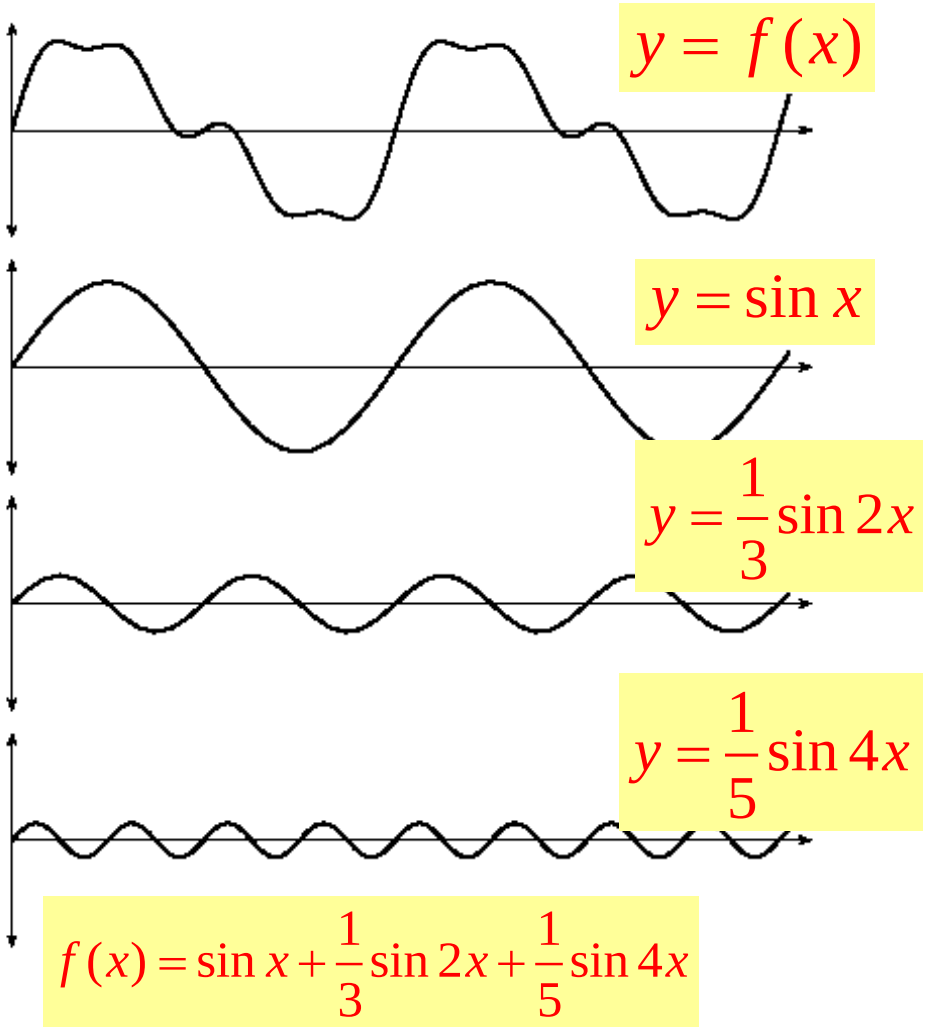
- The Fourier Transform allows us to perform tasks which would be impossible to perform any other way; its efficiency allows us to perform other tasks more quickly.
- The Fourier Transform provides a powerful alternative to linear spatial filtering; it is more efficient to use the Fourier transform than a spatial filter for a large filter.
- The Fourier Transform also allows us to isolate and process particular image frequencies, and so perform low-pass and high-pass filtering with a great degree of precision.



Some Intuition

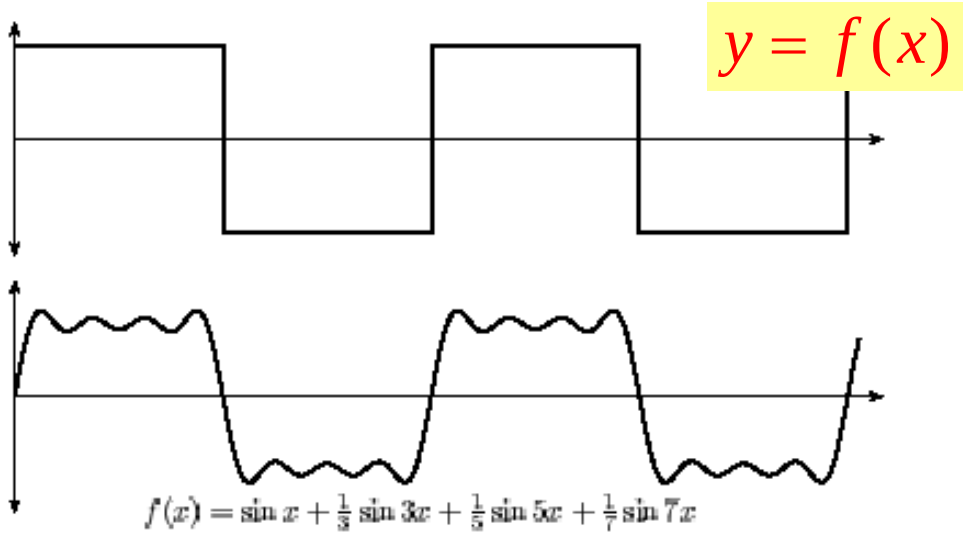
- A periodic function may be written as the sum of sines and cosines of varying amplitudes and frequencies.

Examples =>



Some Intuition

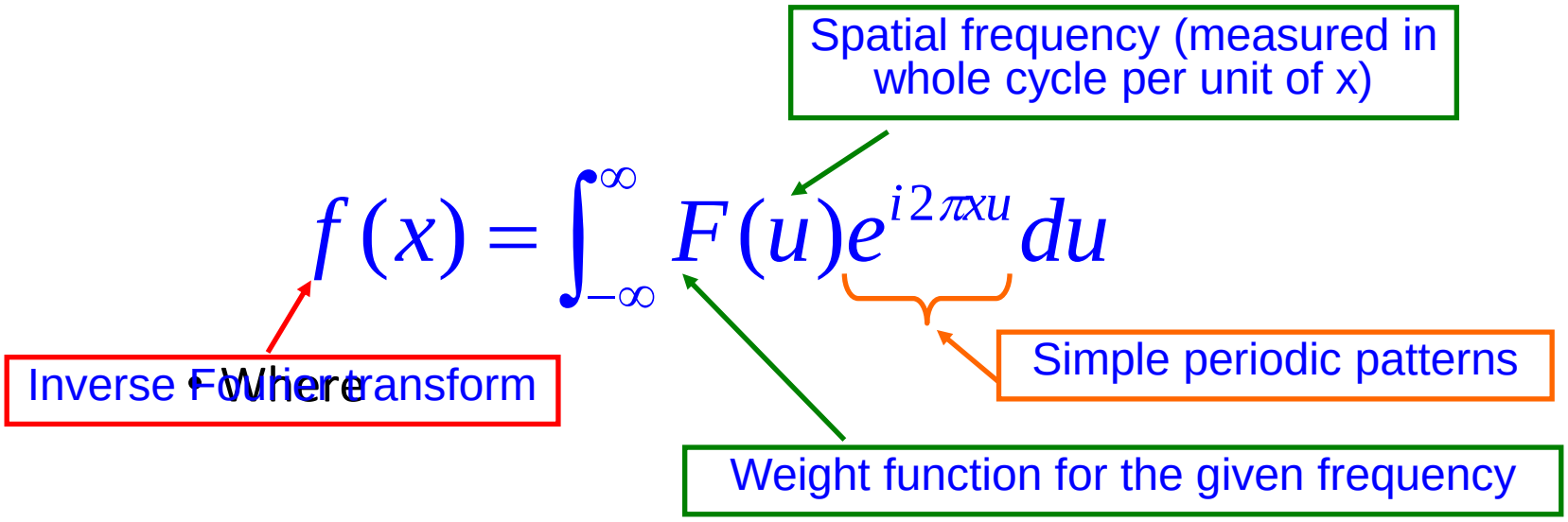
- Some functions will require only a finite number of functions in their decomposition; others will require an infinite number.



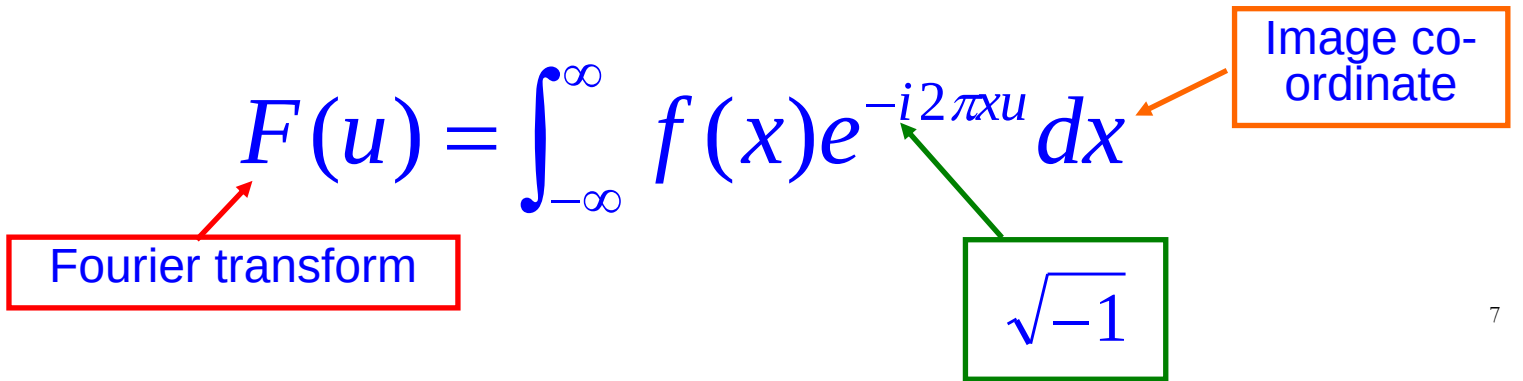
$$f(x) = \sin x + \frac{1}{3}\sin 3x + \frac{1}{5}\sin 5x + \frac{1}{7}\sin 7x + \frac{1}{9}\sin 9x + \ldots$$

1-D Continuous

- f(x) is a linear combination of simple periodic patterns.



- Where

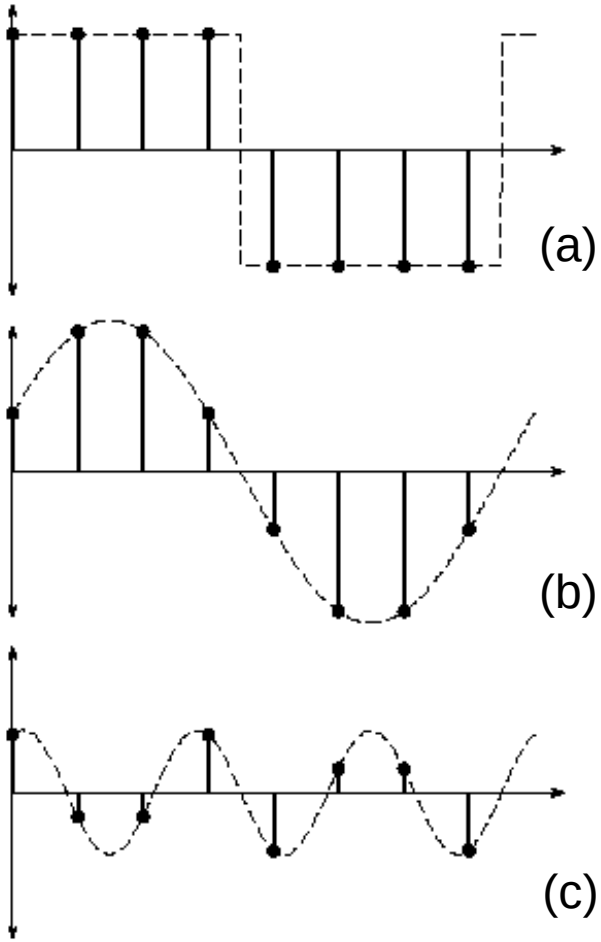


1-D Discrete (con'd)

- In image processing, we deal with a discrete function.
- Since we only have to obtain a finite number of values, we only need a finite number of functions to do it.
- For example: 1 1 1 1 -1 -1 -1 -1,

  which we may take as a discrete approximation to the square wave

  of figure (a). This can be expressed as the sum of two sine functions, (b) and (c)



Definition of 1-D DFT

Suppose $f = [f_0, f_1, f_2, \cdots, f_{N-1}]$

is a sequence of length N. Define its discrete Fourier transform

$$F = [F_0, F_1, F_2, \cdots, F_{N-1}]$$

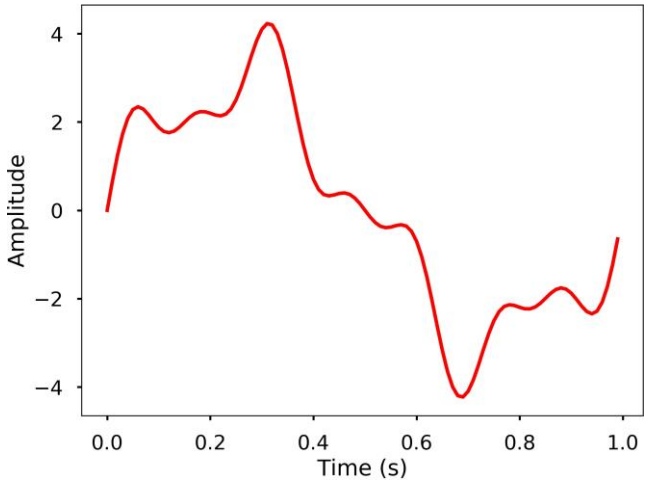
where $$F_u = \frac{1}{N} \sum_{x=0}^{N-1} \exp[-2\pi i \frac{xu}{N}] \cdot f_x$$

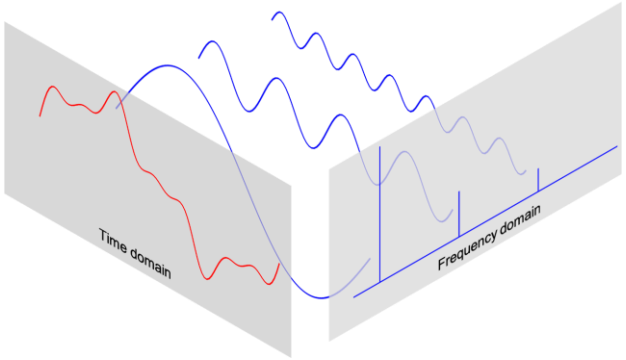
We can express this definition as matrix multiplication

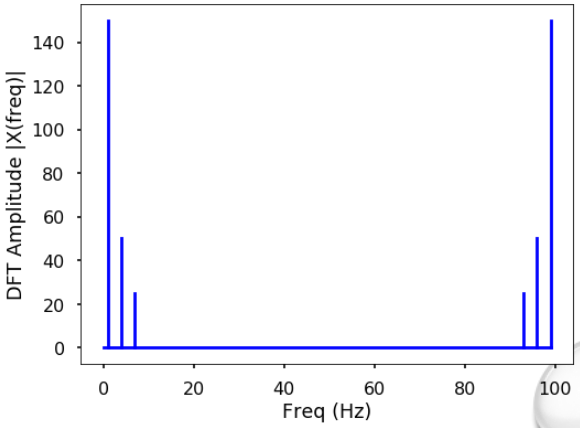
$$F = \mathrm{F} f$$

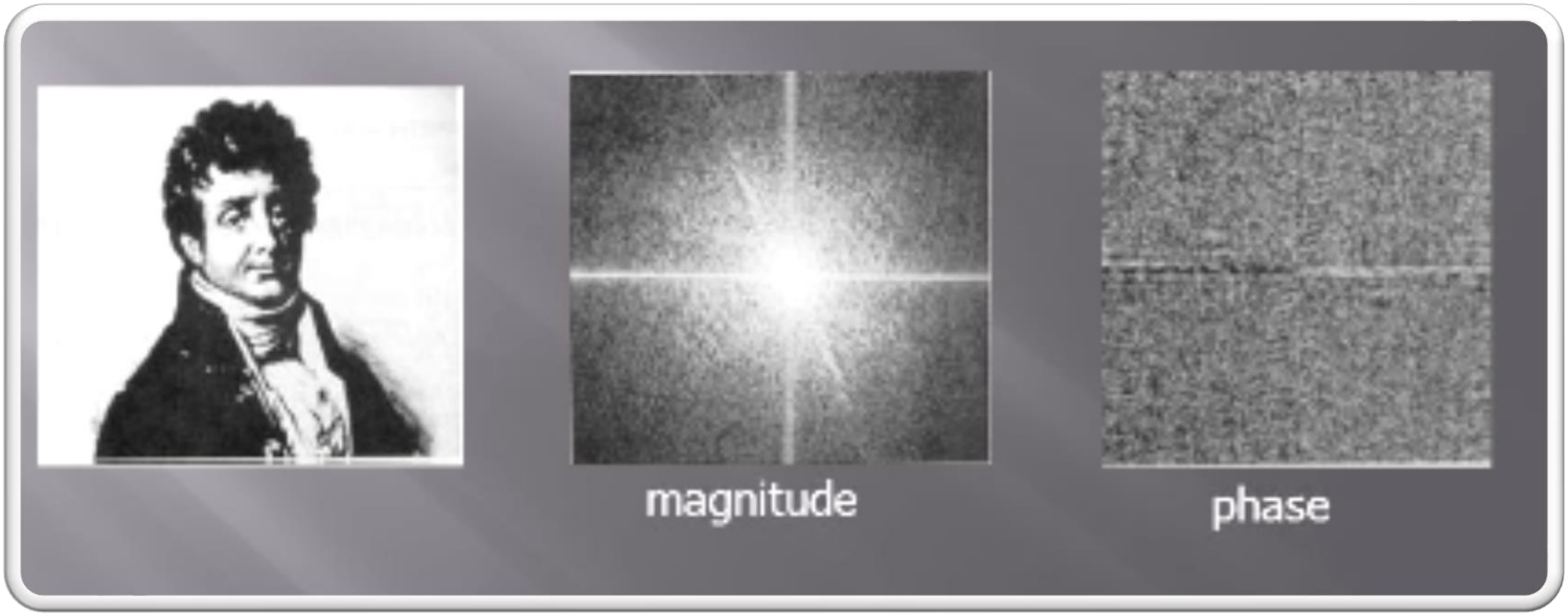
Where F is an NxN matrix defined by $$\mathrm{F}_{m,n} = \frac{1}{N} \exp[-2\pi i \frac{mn}{N}]$$

Example

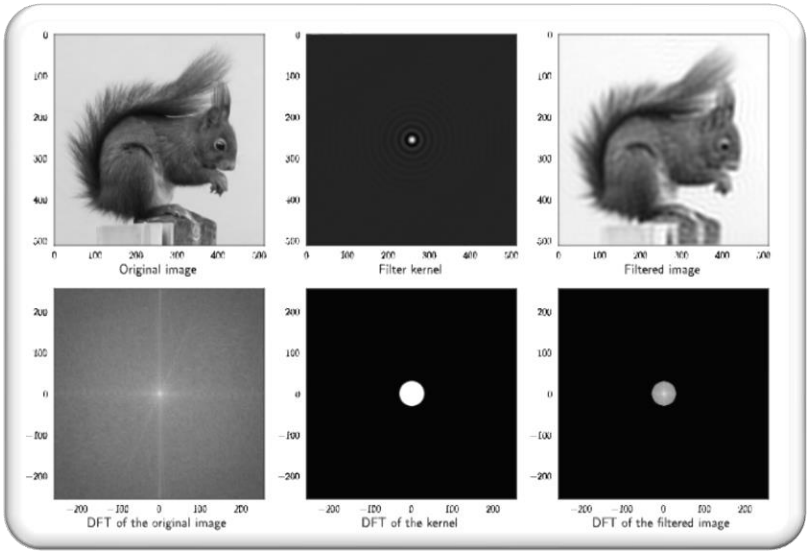

Amplitude
Time (s)
4
2
0
−2
−4
0.0
0.2
0.4
0.6
0.8
1.0


Time domain
Frequency domain


DFT Amplitude |X(freq)|
Freq (Hz)
140
120
100
80
60
40
20
0
0
20
40
60
80
100



2D FFT Example



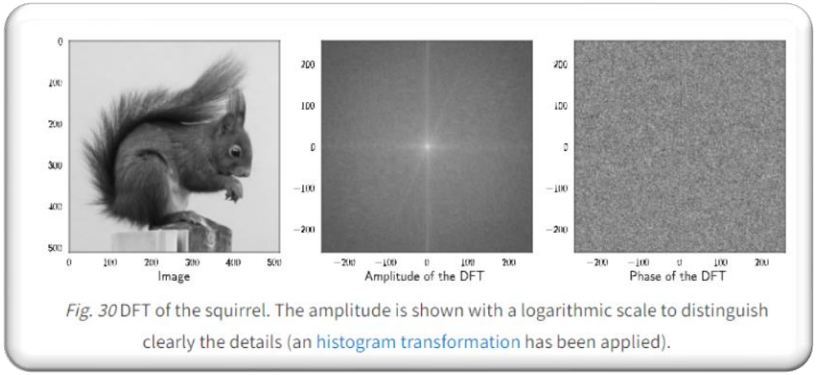


Fig. 30 DFT of the squirrel. The amplitude is shown with a logarithmic scale to distinguish clearly the details (an histogram transformation has been applied).

Low pass filter example

What is the advantage of carrying filtering in the frequency domain over spatial domain?

The primary reason is that in frequency domain, the process of filtering i.e. convolution simplifies to multiplication. Basically the concept of frequency domain mathematics says that given a function $f(x, y)$ and a kernel $g(x, y)$, the filtered output can be written as $f(x, y) * g(x, y)$ where * represents the convolution operator. Convolution can be explained as placing the kernel over the signal and basically summing up the element-wise products and replacing the anchor value with the sum. On the other hand, in frequency domain, the output is simply given by $F(u, v) \times G(u, v)$ where $F(u, v)$ and $G(u, v)$ are the Fourier transforms of their respective functions. This representation is both computationally and spacewise efficient as both may be represented as sparse matrices as well in most cases.

And hence, we use frequency domain for filtering. Generally though, for separable filters, it makes more sense to just use the separated components one at a time.

2-D Discrete

- For an MxN matrix

$$F(u,v) = \frac{1}{MN} \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x,y) \exp\left[-2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right]$$

$$f(x,y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u,v) \exp\left[2\pi i \left(\frac{xu}{M} + \frac{yv}{N} \right) \right]$$

- All 1-D properties transfer into 2-D
- Some more properties useful for image processing.

Separability

$$\exp\left[2\pi i\left(\frac{xu}{M}+\frac{yv}{N}\right)\right]=\exp\left[2\pi i\frac{xu}{M}\right]\exp\left[2\pi i\frac{yv}{N}\right]$$

1st term

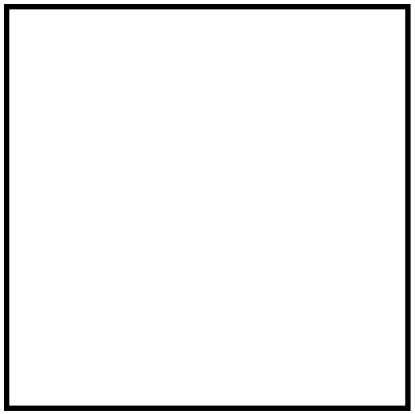
2nd term

- The 1st term depends only on x and u, and the 2nd term depends on y and v.
- Advantage: computing DFT of all the columns first, then computing the DFT of all the rows of the result.

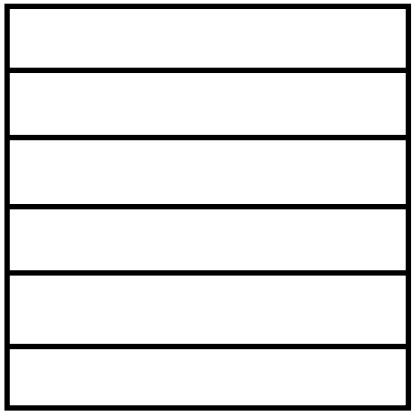
(con'd)

- 1D Fourier pair:

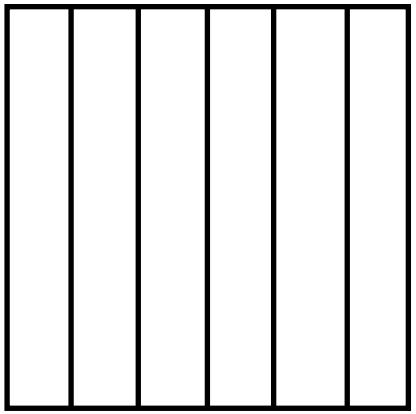
$$F(u)=\frac{1}{M}\sum_{x=0}^{M-1} f(x)\exp\left[-2\pi i\frac{xu}{M}\right];\quad f(x)=\sum_{u=0}^{M-1} F(u)\exp\left[2\pi i\frac{xu}{M}\right]$$



(a) Original image



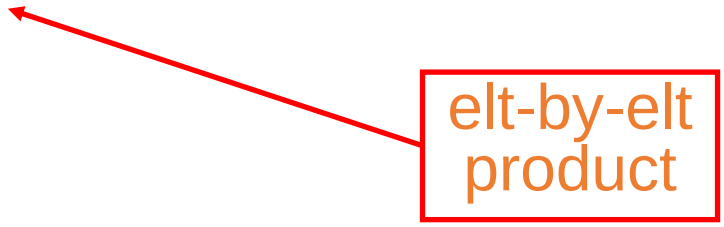
(b) DFT of each row of (a), using x & u



(c) DFT of each column of (b), using y & v

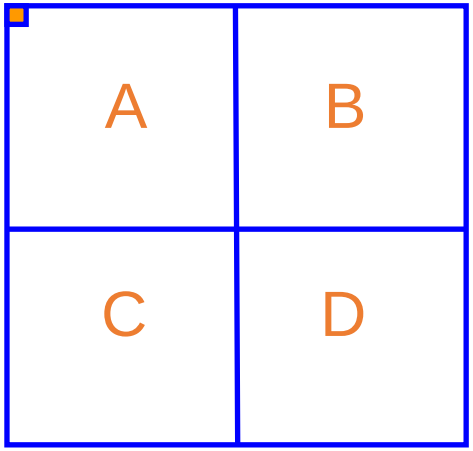
Convolution Theory

- Review:
  - How to convolve an image M with a spatial filter S in spatial domain?
  - If Z,X,Y are the DFT's of z=x*y, x and y respectively, then Z=X.Y (convolution theory)
- We can perform the same operation (convolution) in frequency domain
  - Pad S with 0, so same size as M; denote the new matrix S'
  - Perform DFT on M and S' to obtain F(M) and F(S')
  - Perform inverse DFT on F(M).F(S') to get F^{-1}
  - M*S is F^{-1}
- Great saving for a large filter.

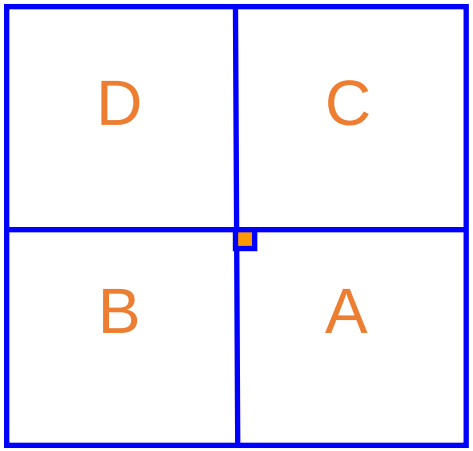


Shifting

- Review: In 1-D, if multiplying each element x_n of vector x by $(-1)^n$ we swap the left and right halves of the Fourier transform.
- In 2-D, the same principle applies if multiplying all elements $x_{m,n}$ by $(-1)^{m+n}$ before the transform.



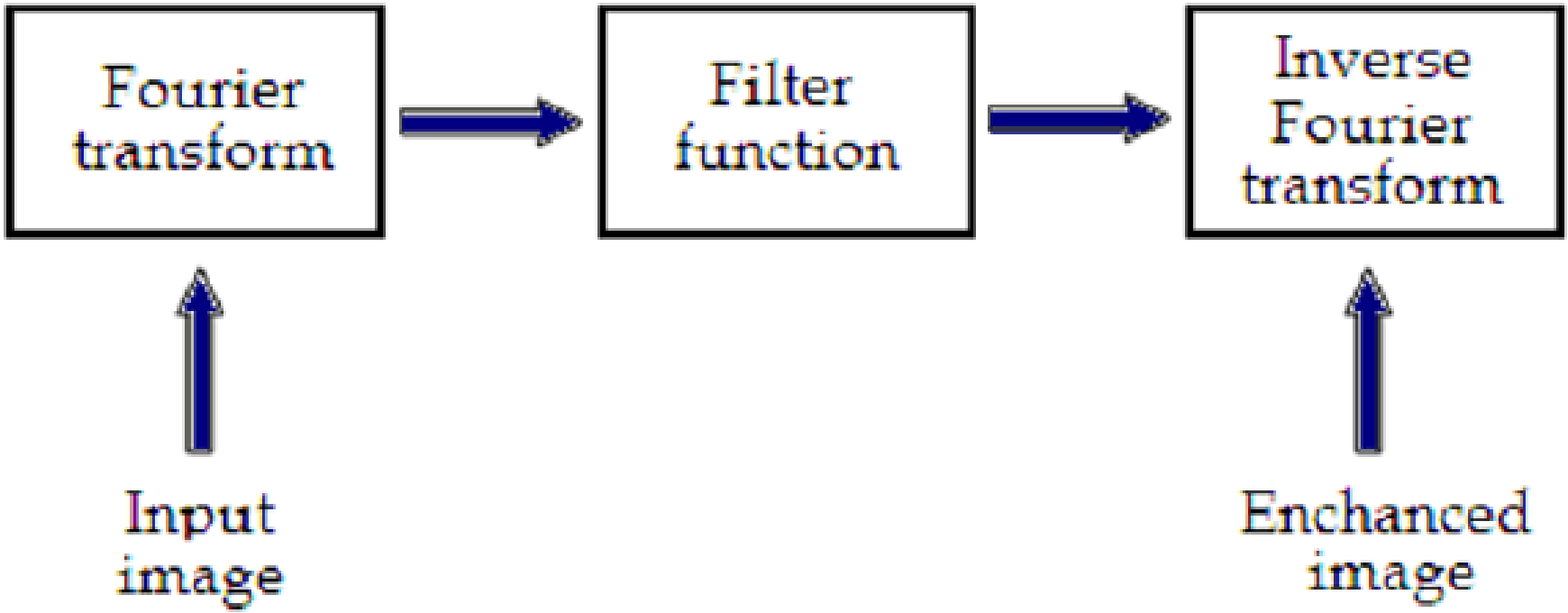
An FFT



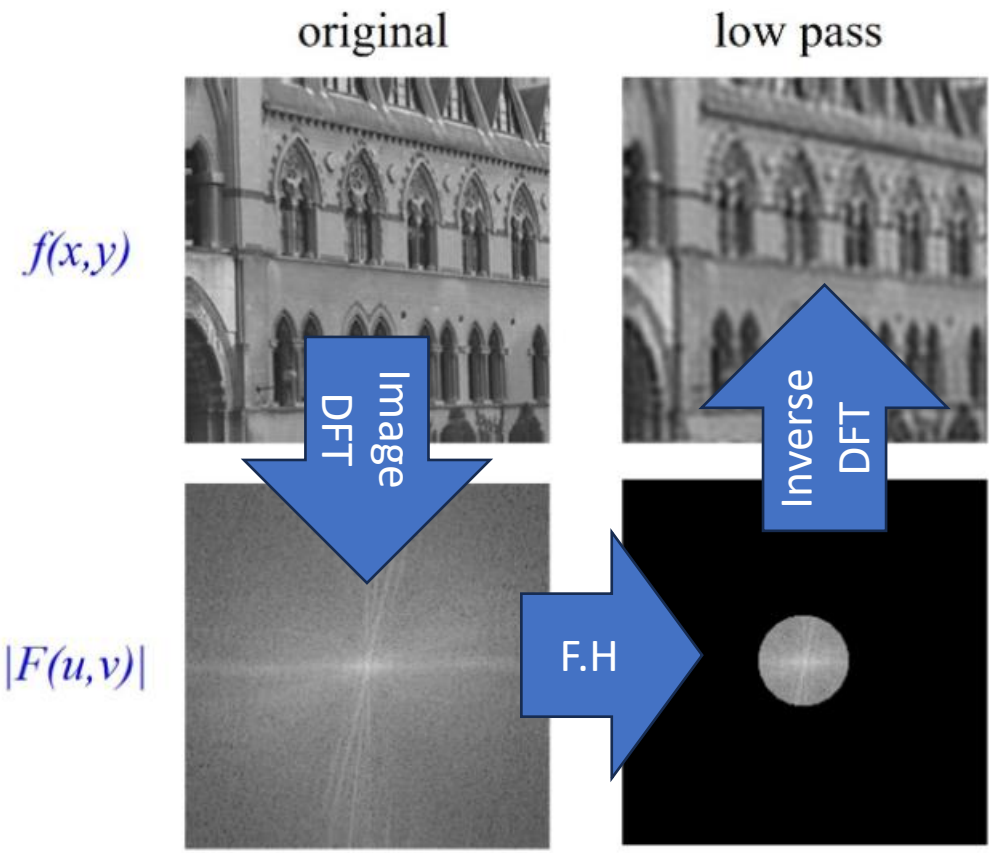
After shifting

Frequency filters process

- Frequency filters process an image in the frequency domain. The image is Fourier transformed, multiplied with the filter function and then re-transformed into the spatial domain. Attenuating high frequencies results in a smoother image in the spatial domain, attenuating low frequencies enhances the edges.



Example



Filtering

- Our basic model for filtering in the frequency domain is

$$G(u,v) = H(u,v)F(u,v)$$

result | filter | Fourier transform of image to be smoothed

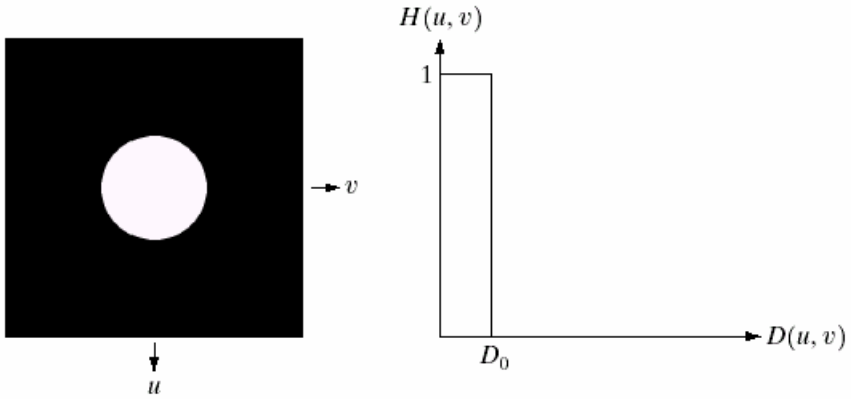
- We'll briefly discuss 3 types of filters in the order of increasing smoothness:
  - Ideal
  - Butterworth
  - Gaussian
- Preprocessing: F is shifted so that the DC coefficient is in the center.

Ideal Filtering:Low-Pass (ILPF)

- The low-frequency components are toward the center.
- Multiplying the transform by a matrix to remove or minimize the values away from the center.
- Ideal low-pass matrix *H*

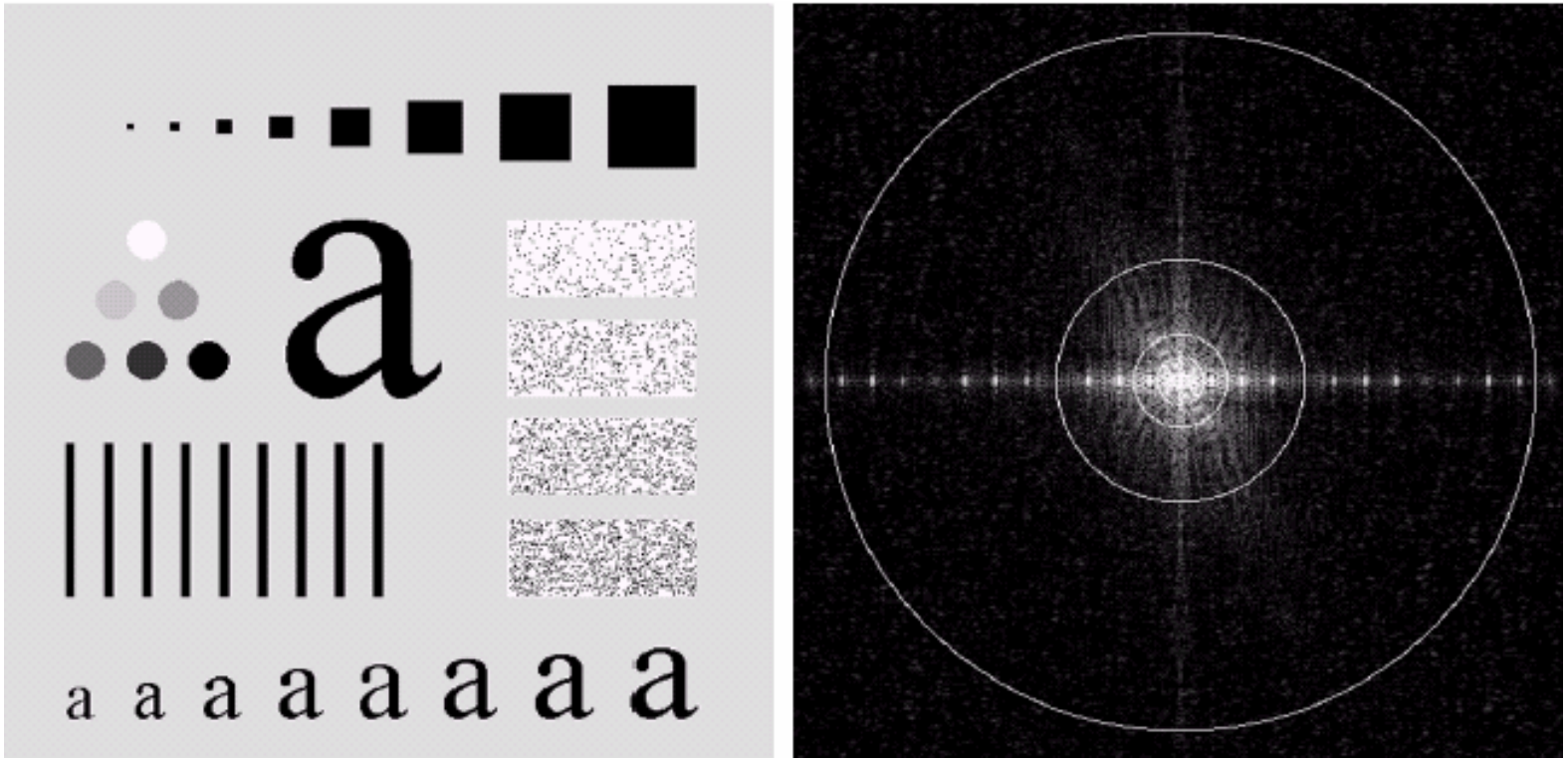
$$H(u,v) = \begin{cases} 1 & \text{if } D(u,v) \le D_0 \\ 0 & \text{otherwise} \end{cases}$$

- The inverse DFT of *H.F* is the smoothed image.



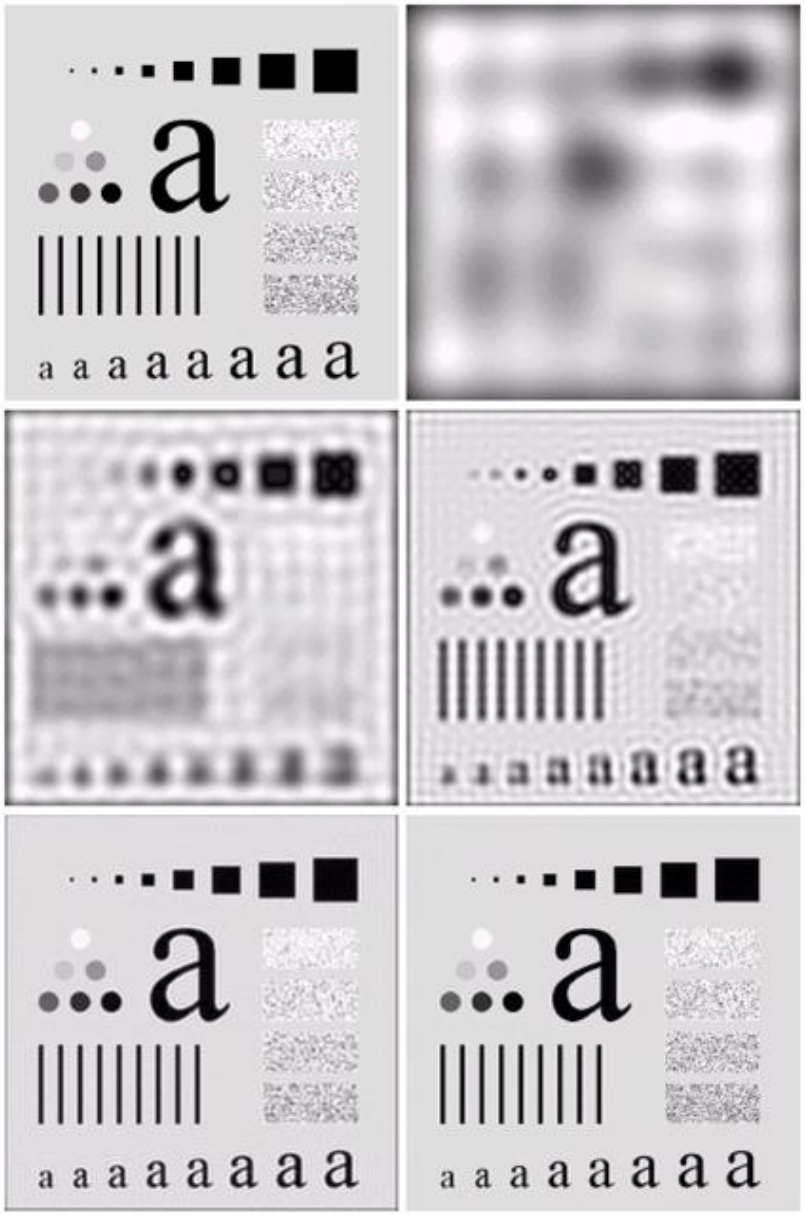
D(u,v) is distance from the origin of the Fourier transform, shifted to the center

Demo



An image with its Fourier spectrum. The superimposed circles have radii of 5, 15, 30, 80, and 230

Results



D=5

D=15

D=30

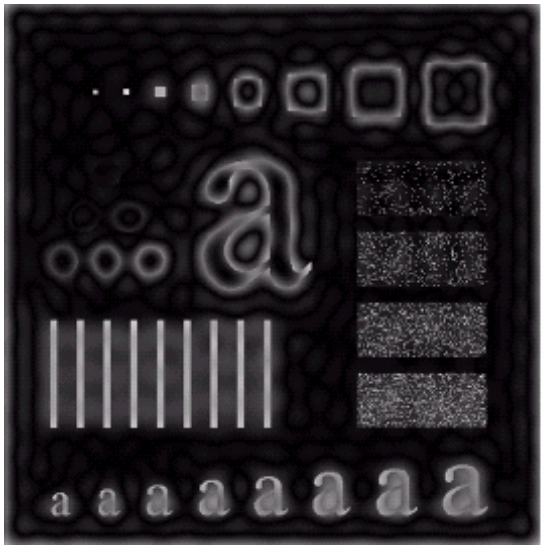
D=80

D=230

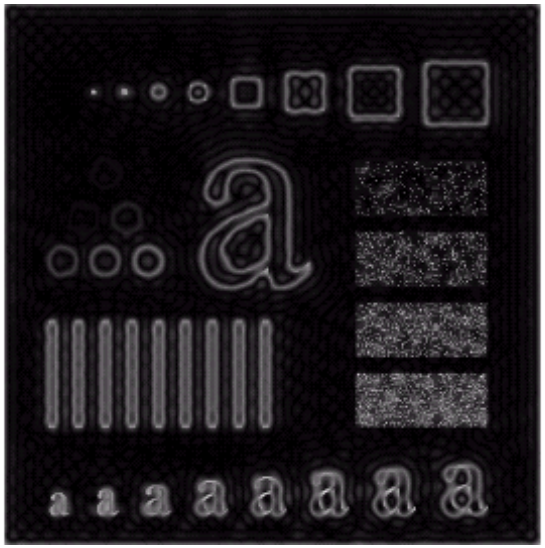
Ideal Filtering:High-Pass (IHPF)

- Opposite to low-pass filtering: eliminating center and keeping the others.

$$H(u,v) = \begin{cases} 0 & \text{if } \mathrm{D}(u,v) \le \mathrm{D}_0 \\ 1 & \text{otherwise} \end{cases}$$



D_0=15



D_0=30



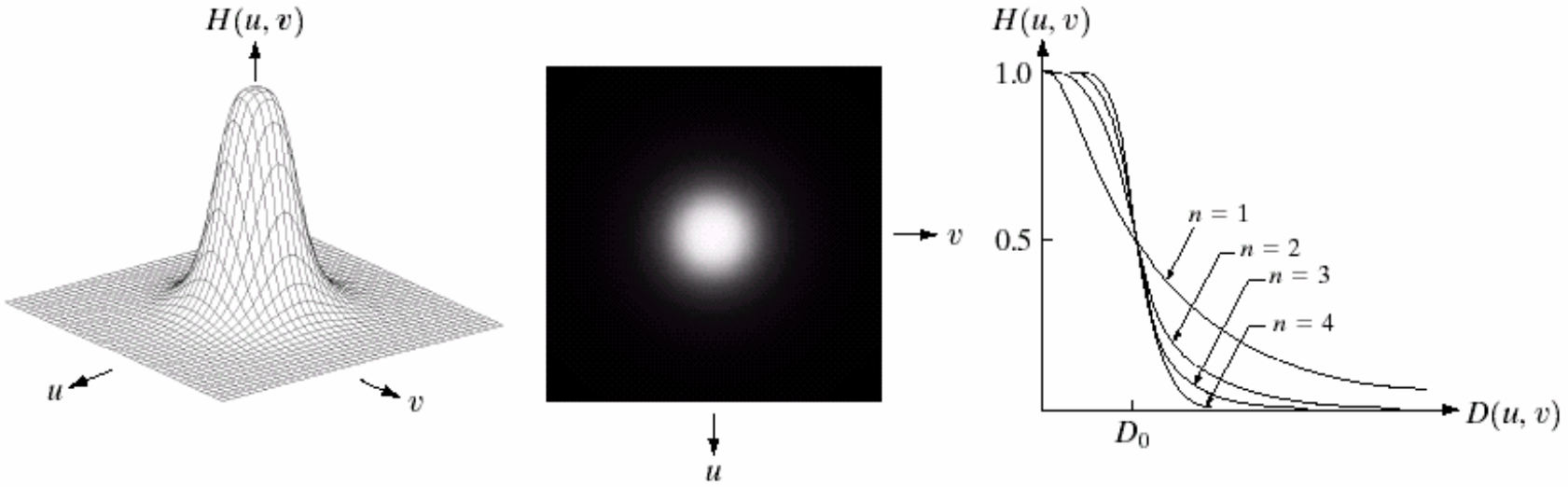
D_0=80

Butterworth Filtering: Low-Pass (BLPF)

- Unlike ILPF, BLPF does not have a clear cutoff between passed and filtered frequencies.

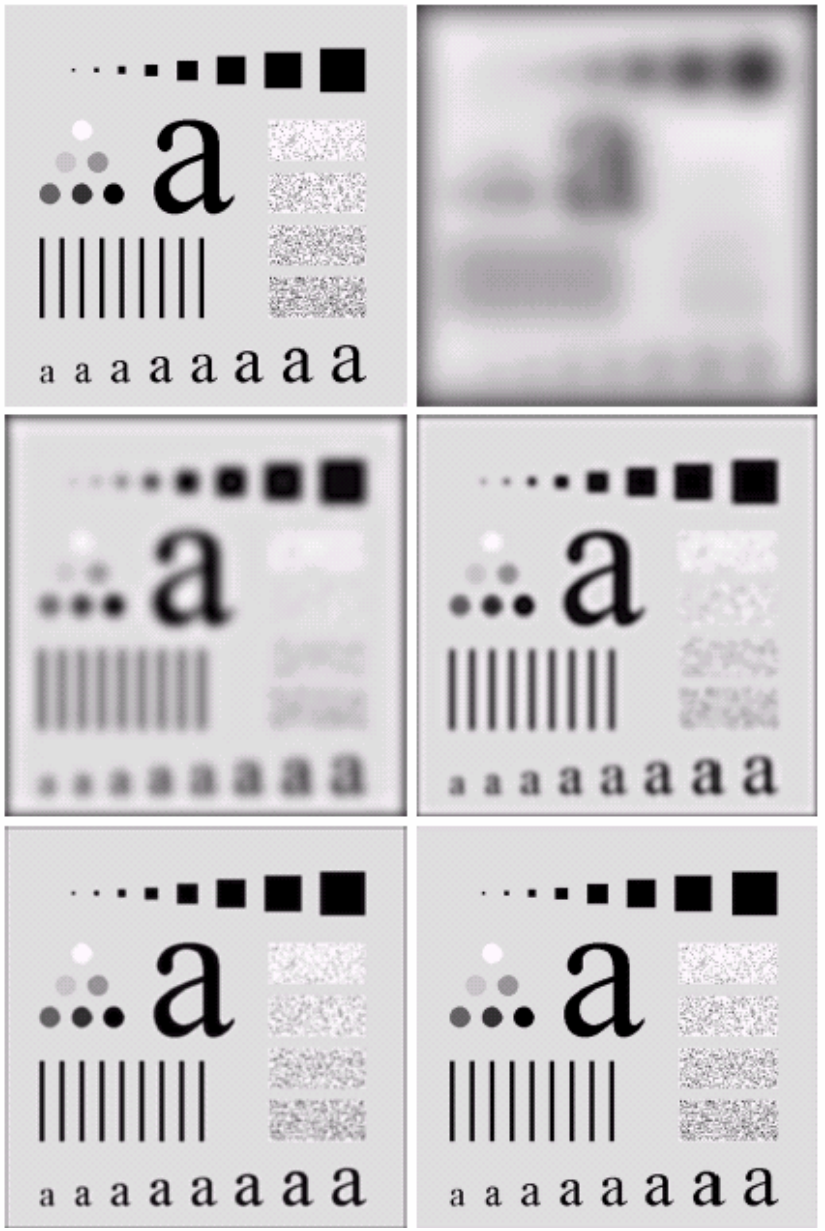
$$H(u,v) = \frac{1}{1+\left[D(u,v)/D_0\right]^{2n}}$$

n is the order of the filter



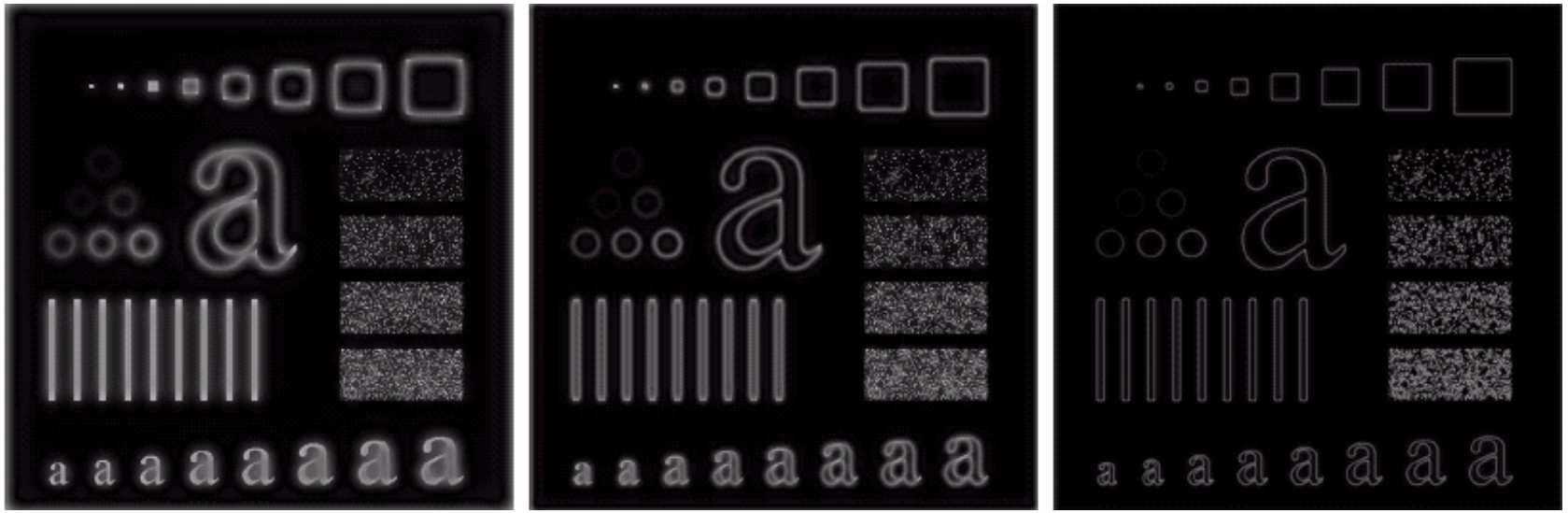
Demo

- n=2 and D_0 equal the 5 radii



Butterworth Filtering: High-Pass (BHPF)

$$H(u,v) = \frac{1}{1+\left[D_0 / D(u,v)\right]^{2n}}$$



D_0=15 D_0=30 D_0=80

Gaussian Filtering: Low-Pass (GLPF)

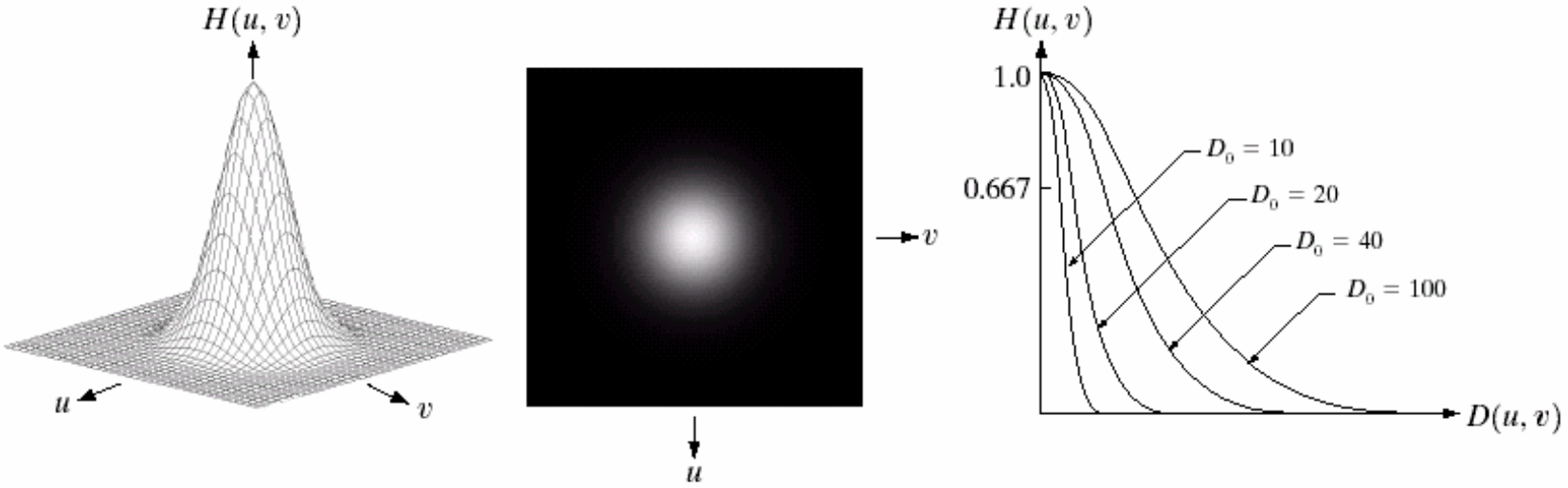
- We mentioned Gaussian once in the section for spatial filtering.
- Will discuss more in detail in image restoration.
- Gaussian filter in frequency domain:

$$H(u,v) = e^{-D^2(u,v)/2\sigma^2}$$

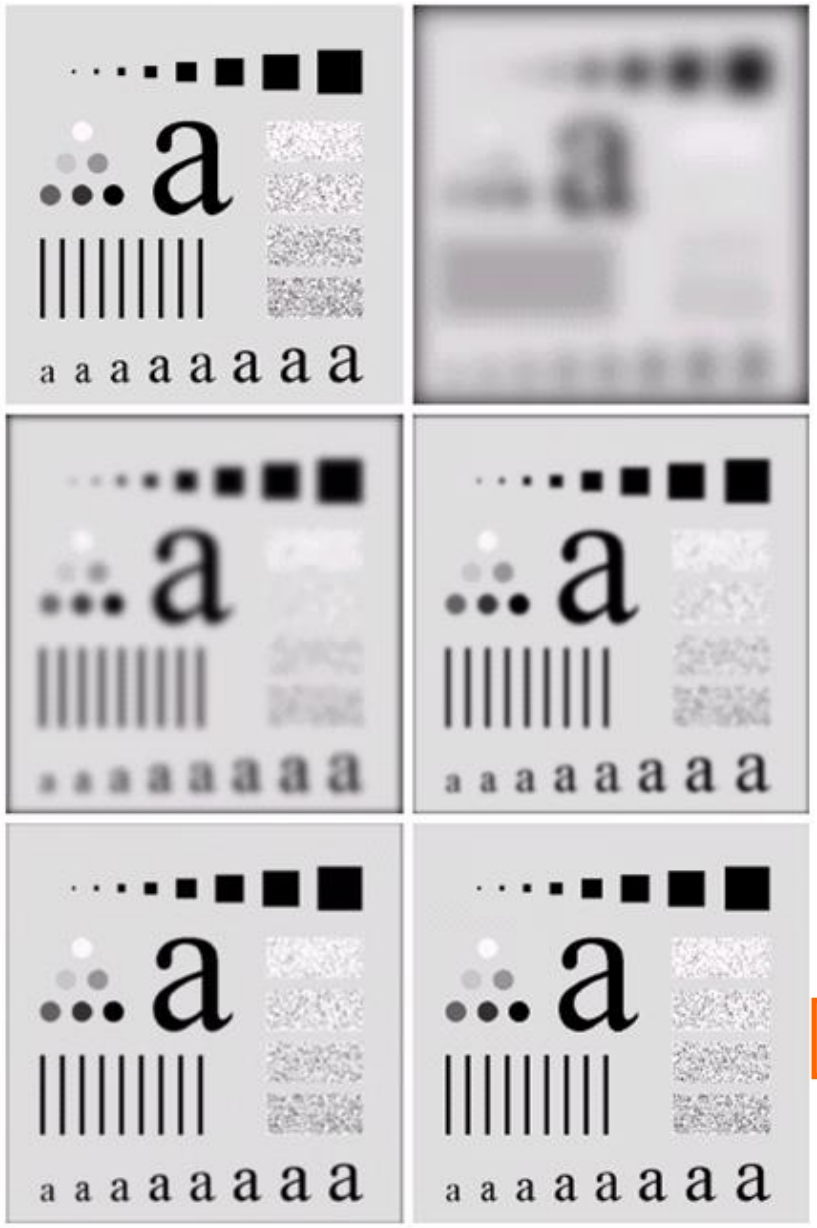
- The inverse is also a Gaussian.
- We may replace σ by D_0, which is the cutoff frequency.

$$H(u,v) = e^{-D^2(u,v)/2D_0^2}$$

GLPF (con'd)



Demo



D_0=5

D_0=15

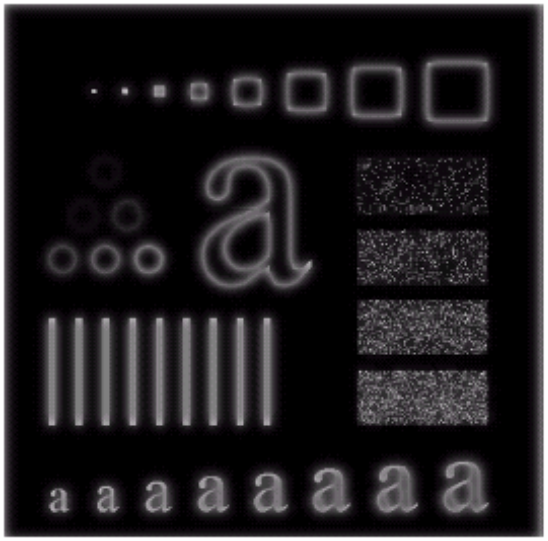
D_0=30

D_0=80

D_0=230

Gaussian Filtering: High-Pass (GHPF)

$$H(u,v) = 1 - e^{-D^2(u,v)/2D_0^2}$$



D_0=15

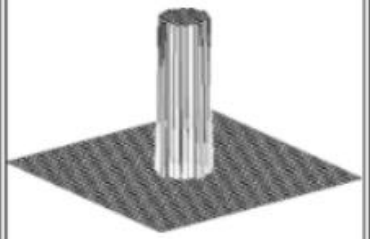
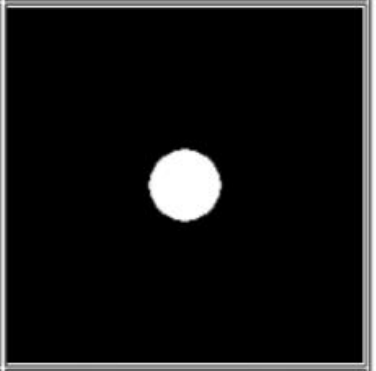
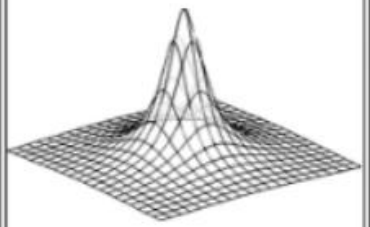
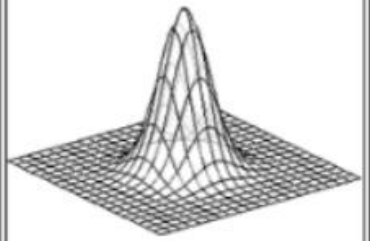
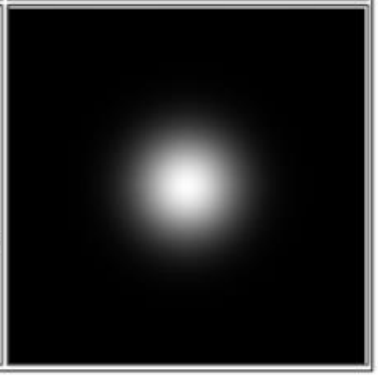


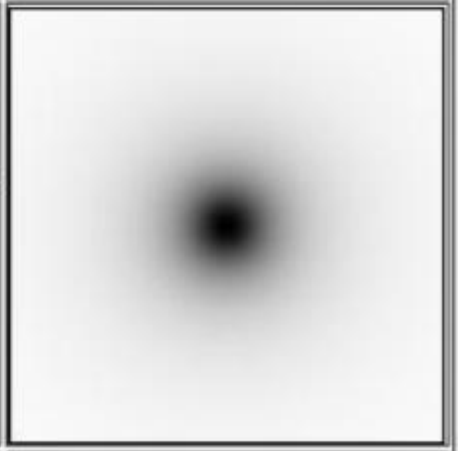
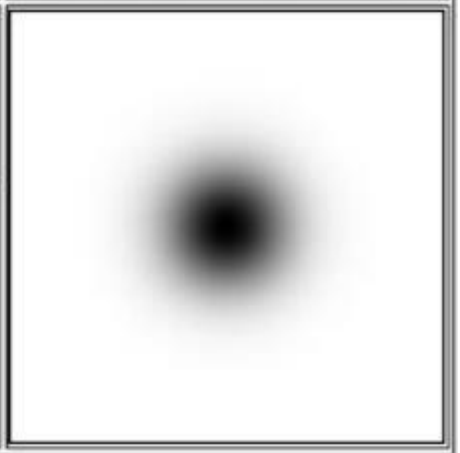
D_0=30



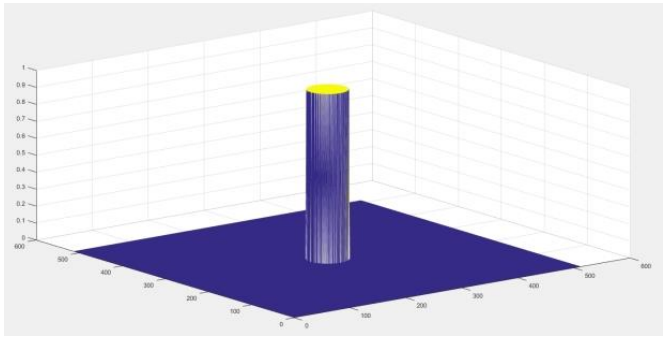
D_0=80


REMEMBER

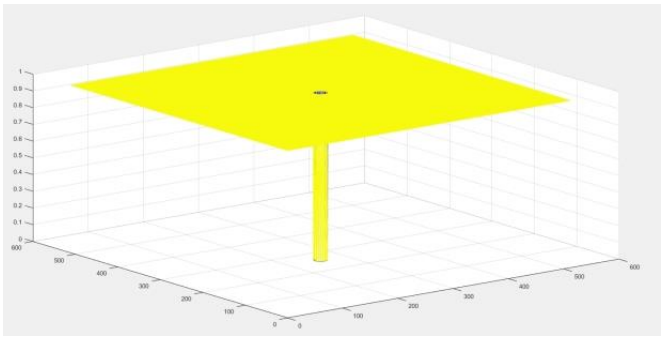
Lowpass Filter	Mesh	Image
Ideal: $H(u,v)=\begin{cases}1 & \text{if } D(u,v)\le D_0\\ 0 & \text{if } D(u,v)>D_0\end{cases}$		
Butterworth: $H(u,v)=\frac{1}{1+\left[D(u,v)/D_0\right]^{2n}}$		
Gaussian: $H(u,v)=e^{-D^2(u,v)/2D_0^2}$		

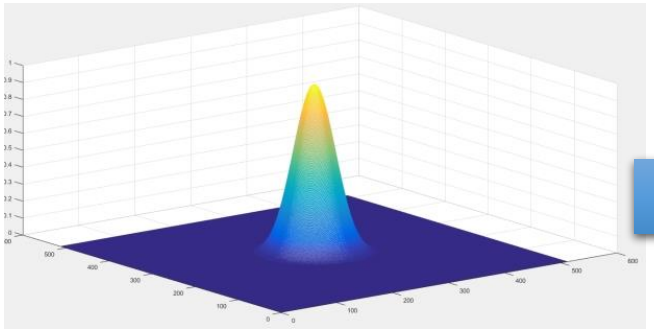
Highpass	Mesh	Image
Ideal		
Butterworth		
Gaussian		

Low pass Transfer Function	Hight pass Transfer function

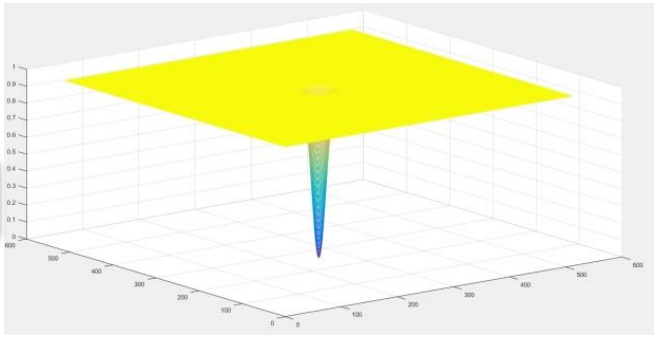


Ideal





Gaussian



Steps to Apply Filtering

Step 1: Input – Read an image

Step 2: Saving the size of the input image in pixels

Step 3: Get the Fourier Transform of the input_image

Step 4: Assign the Cut-off Frequency D_o

Step 5: Designing filter: High Pass Filter or low pass filter

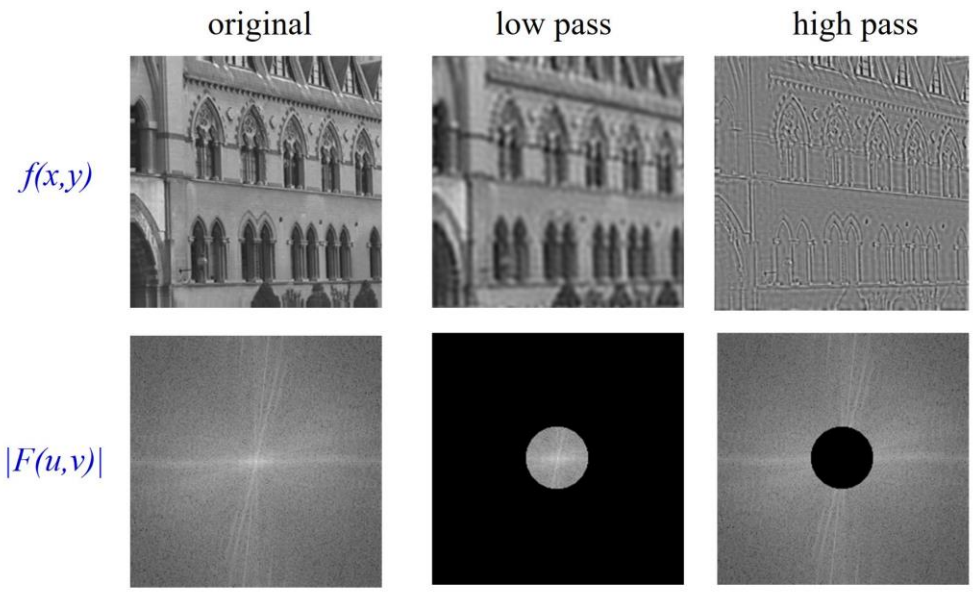
Step 6: Multiply Fourier Transformed input image and the filtering mask

Step 7: Take Inverse Fourier Transform of the convoluted image

Step 8: Display the resultant image as output

Example

Example: action of filters on a real image



In-class Exercise

- Q. What is the Fourier transform of the average filtering using the 4 immediate neighbors of point (x,y), but excluding itself?

$$g(x,y)=\frac{1}{4}\left[f(x,y+1)+f(x+1,y)+f(x-1,y)+f(x,y-1)\right]$$

Summary

- To perform filtering in frequency domain, do the following steps:
  - The Fourier transform of the image is shifted, so that the DC coefficient is in the center. (multiply the image by $(-1)^{x+y}$)
  - Create the filter
  - Multiply it by the image transform
  - Invert the result
  - Multiplying the result by $(-1)^{x+y}$.
- The relationship between the corresponding high- and low-pass filters:

$$H_{hp}(u,v) = 1 - H_{lp}(u,v)$$